

Clase 1: Fundamentos de Ingeniería de Software (IS)

CETP-UTU · CTT · Módulo I - Técnico en Redes y Software

Duración Total: 120 minutos (Teoría) + 40 minutos (Práctica / Laboratorio)

Unidad 1: Conceptos de la Ingeniería de Software

Fuentes y Bibliografía de Referencia

- **El Cuerpo de Conocimiento:** IEEE SWEBOK v4.0 (*Software Engineering Body of Knowledge*).
- **Ingeniería Clásica:** Pressman, R. S. *"Ingeniería del software: un enfoque práctico"*.
- **Estándares SDLC y Requerimientos:** ISO/IEC/IEEE 12207 e ISO/IEC/IEEE 29148:2018.
- **Seguridad (AppSec):** ISO/IEC 27034 y marco de trabajo OWASP.
- **Filosofía y Calidad:** *"El Manifiesto Ágil"* (Fowler, Beck, et al.) y *"Clean Code"* (Robert C. Martin).
- **Ingeniería Ágil:** Beck, K. *"Extreme Programming Explained: Embrace Change"*.

I. Introducción: SWEBOK y el Factor Humano (25 min)

1. El Mito del Programador y la Profesionalización

Bienvenidos a Ingeniería de Software. Históricamente se creía que nuestra carrera trataba simplemente de "sentarse a escribir código rápido". La realidad es que entre el 60% y el 80% de los proyectos IT fracasan por problemas de gestión, mala comunicación o falta de diseño.

Para dejar de ser "picadores de código" y convertirnos en profesionales, la industria creó el **SWEBOK v4.0**. Este documento define las áreas de conocimiento que todo ingeniero debe dominar: no solo construir, sino planificar, diseñar, probar y mantener.

2. El Impacto Humano de un Mal Software

Para entender por qué aplicamos ingeniería, no necesitamos irnos a ejemplos de ciencia ficción o tragedias letales. El mal software afecta la vida diaria de formas devastadoras.

- **El Caso Horizon (Correo Británico - Años 2000/2010):** El sistema informático *Horizon*, utilizado por las sucursales del correo, tenía bugs (errores de código) que calculaban mal los saldos diarios, mostrando siempre "faltantes de dinero" que no eran reales. En lugar de investigar el software (falta de pruebas y auditoría), la empresa culpó a los encargados de las sucursales.

- **El costo humano:** Más de 700 personas inocentes fueron procesadas por robo; muchos perdieron sus casas para pagar "deudas" inexistentes, sus familias se destruyeron y varios terminaron en prisión.
- **La Lección:** El software gestiona becas, sueldos, historiales médicos y ahorros. Un error no controlado en nuestro código o un relevamiento mal hecho no es "solo un bug", es una familia que no cobra su sueldo a fin de mes. La IS nos da la ética y la metodología para evitar esto.



II. El Ciclo de Vida (SDLC) y el *Shift-Left* (25 min)

Para construir sistemas robustos que no arruinen vidas, pasamos por una línea de montaje llamada **SDLC** (*Software Development Life Cycle* / Ciclo de Vida del Desarrollo de Software).

Las Fases y el Paradigma de Seguridad

Hoy la industria mundial aplica el paradigma **Shift-Left** (Mover a la izquierda): esto significa inyectar seguridad y pruebas de calidad desde el día uno de trabajo. Descubrir una falla de cálculo en un sueldo durante el *Diseño* cuesta 1 dólar; descubrirla cuando ya se le depositó mal a 10.000 empleados en *Producción* cuesta cientos de miles.

1. **Relevamiento (Ingeniería de Requerimientos - ISO 29148):** Hablar con el usuario, entender su problema real.
2. **Análisis y Diseño:** Modelar la solución en planos. Aquí aplicamos *Security by Design* (Seguridad desde el Diseño).
3. **Implementación:** Escribir el código aplicando **Clean Code** (Código Limpio). El código se lee 10 veces más de lo que se escribe; si lo dejas desordenado, el equipo que venga después no podrá mantenerlo.
4. **Pruebas (QA - Quality Assurance):** Buscar intencionalmente bugs, fallas de lógica y vulnerabilidades antes de que el cliente lo vea.
5. **Despliegue y Mantenimiento:** Instalar el sistema en servidores y darle soporte continuo.



III. Los 3 Pilares del Modelado UML (25 min)

En la fase de Análisis y Diseño, no "imaginamos" el sistema en el aire, dibujamos planos. El estándar universal para esto es **UML** (*Unified Modeling Language*). Existen muchos diagramas, pero todo técnico debe dominar estos tres fundamentales:

1. Diagramas de Casos de Uso (El Comportamiento / El Alcance)

Nos dice *qué* hace el sistema y *quién* lo usa. No le importa cómo está programado por dentro.

- **Límite del Sistema:** Qué está adentro y qué queda afuera.
- **Casos de Uso (Óvalos):** Funcionalidades (Ej: "Pagar Factura").
- **Actores:** ⚠ *La Regla de Pressman:* "Un actor es cualquier entidad que interactúa con el sistema, pero que NO es parte de él". Puede ser un usuario humano (Cajero), o un sistema externo (API del Banco).

2. Diagramas de Clases (La Estructura Lógica)

Es el corazón de la programación orientada a objetos. Modela *qué datos* guarda el sistema y cómo se relacionan.

- Ejemplo: Una clase Alumno que tiene atributos como Nombre y Cédula, y métodos como Inscribirse(). Es el paso previo a crear la base de datos.

3. Diagramas de Implementación / Despliegue (La Estructura Física)

Nos muestra *dónde* va a vivir nuestro código. Modela los servidores, la nube, las computadoras de los usuarios y cómo se conectan a través de la red (Ej: Un servidor de Base de Datos conectado por protocolo seguro TCP/IP a un Servidor Web).



IV. Modelos de Trabajo: Ágil y Extreme Programming (25 min)

¿Cómo organizamos las fases del SDLC en el calendario?

1. El Riesgo de la Cascada (Waterfall)

El modelo tradicional dicta terminar una fase para empezar la otra. *El problema:* Si relevamos mal los requisitos en enero, y no probamos el software hasta diciembre, el fracaso es total y el presupuesto ya se gastó.

2. El Manifiesto Ágil

Para solucionar esto, nace en 2001 el **Manifiesto Ágil**, priorizando:

- **Individuos e interacciones** sobre procesos y herramientas rígidas.
- **Software funcionando** sobre documentación interminable.
- Trabajamos en "Iteraciones" (ej: ciclos de 2 semanas) para mostrar avances constantes al cliente y corregir el rumbo a tiempo.

3. Extreme Programming (XP)

Es la metodología ágil más técnica del mercado, enfocada en la calidad absoluta del código (evitar casos como el de Horizon).

- **Pair Programming (Programación en Parejas):** Dos personas en una computadora. Uno escribe la táctica (*Driver*), el otro analiza la estrategia y posibles errores (*Navigator*).
- **TDD (Test-Driven Development):** Se escribe la prueba automática *antes* que el código. Si la prueba falla, el código no sirve.
- **Refactoring:** Se limpia y mejora la estructura interna del código constantemente para que no envejezca mal.



V. Gestión Profesional: ALM y GitHub Projects (20

min)

La Ingeniería de Software requiere organizar quién hace qué. A esto se le llama **ALM** (*Application Lifecycle Management*). Nuestra herramienta oficial será **GitHub Projects**.

Prerrequisito Innegociable: Para ser parte de la comunidad de desarrollo, **todos deben tener una cuenta creada en GitHub.com**. Es su currículum digital.

El Tablero Kanban en GitHub:

No usamos cuadernos ni Excel. Usamos un tablero visual donde cada funcionalidad se convierte en un **Issue** (Incidencia/Tarea) que fluye por columnas:

1. **Backlog (Pila de Producto):** Todo lo que hay que hacer a futuro.
2. **Todo (Por hacer hoy/esta semana):** Tareas asignadas al equipo.
3. **In Progress (En progreso):** Lo que estamos programando ahora mismo.
4. **In Review / QA:** Código terminado, esperando que un compañero lo revise.
5. **Done (Hecho):** Probado y seguro.



VI. Laboratorio Práctico (40 min)

Dinámica Progresiva: Todos inician en el *Nivel Base*. Quienes lo dominen y terminen, avanzarán a *Profundización*.

Contexto del Proyecto Semestral: Construir el "Sistema de Gestión de Alumnos del CTT".



Parte 1: Nivel Base (Todos los estudiantes)

1. **El Factor Humano:** Describe brevemente un error de software cotidiano (que hayas vivido o imaginado, como un doble cobro en el ómnibus o una materia que desapareció de tu escolaridad) y explica a qué fase del SDLC crees que se debió.
2. **Identificación de Actores (UML):** Basándote en Pressman, identifica 3 **Actores** para el Sistema del CTT (recuerda incluir al menos un sistema externo).
3. **UML General:** De los 3 diagramas aprendidos (Casos de Uso, Clases, Implementación), ¿cuál usarías para mostrarle al Director del CTT cómo el sistema se conectará al servidor de la UTU?



Parte 2: Nivel Profundización (Avanzados)

1. **Clasificación RF vs RNF:**
 - "El sistema debe encriptar las contraseñas." -> _____
 - "El docente registrará la asistencia por fecha." -> _____
2. **Aplicando Shift-Left:** Escribe una pregunta de *Relevamiento* que le harías al Director del CTT para prevenir que un alumno pueda ver las notas de otro compañero.
3. **XP en la vida real:** ¿Por qué crees que el *Pair Programming* (Programación en parejas) podría haber evitado una tragedia contable como la del Sistema Horizon del Correo

Británico?

Parte 3: Setup de Entorno (En Equipos)

Objetivo: Integrarse al estándar de la industria.

1. **Individual:** Crear cuenta personal en GitHub.com (si no la tienen).
2. **Grupal:** Uno del equipo crea un repositorio llamado SGA-CTT (Sistema Gestión Alumnos) y agrega a sus compañeros como colaboradores.
3. Vayan a la pestaña **Projects** del repositorio y creen un tablero Kanban básico.
4. Creen las columnas: Backlog, In Progress, Review, y Done.
5. Transformen 2 funcionalidades pensadas para el sistema en **Issues** y déjenlas en su Backlog.